TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐẠI HỌC ĐÀ NẴNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CƠ SỞ (PBL2)

HỆ THỐNG MÔ PHỎNG & ĐIỀU PHỐI TÀU HỎA DÂN DỤNG TP. ĐÀ NẴNG

(Bi-directional Double Track, Priority Dijkstra, Block Signaling & Dynamic Re-routing)

Sinh viên thực hiện: niaagngi07

Email: nghialee1904@gmail.com

Công nghệ nền tảng: C++17, Raylib 5.0, CMake, MinGW UCRT64

Mã nguồn dự án: https://github.com/niaagngi07/PBL2.git

Thời gian hoàn thành: Tháng 11 / 2026

---

1. TỔNG QUAN HỆ THỐNG & CÁC CHỨC NĂNG CỐT LÕI

Dự án **Hệ thống Điều phối Tàu hỏa Dân dụng TP. Đà Nẵng** được xây dựng nhằm giải quyết bài toán mô phỏng, lập lịch trình, giám sát và điều phối giao thông đường sắt đô thị theo thời gian thực trên nền bản đồ số hóa tỷ lệ chuẩn của TP. Đà Nẵng. Hệ thống vận hành theo kiến trúc hướng đối tượng kết hợp phân tầng chuẩn công nghiệp (Layered Architecture / MVC), tích hợp trực quan hóa đồ họa 2D tốc độ cao bằng thư viện Raylib.

1.1. Quy mô mạng lưới đường sắt quy hoạch TP. Đà Nẵng

- **02 Tuyến MRT Cao Tốc (Mass Rapid Transit):** MRT Tuyến xương sống vận tải khối lượng lớn, vận tốc tối đa **80 km/h**, **Mức ưu tiên 1 (Cao nhất)**. Gồm MRT 1 (Trục Tây Bắc - Liên Chiểu - Sân Bay - Sông Hàn) và MRT 2 (Bờ Đông Sông Hàn - Sơn Trà - Ngũ Hành Sơn - Hội An).
- **11 Tuyến LRT Đô Thị (Light Rail Transit):** LRT Tuyến đường sắt nhẹ kết nối nội đô và các khu dân cư vệ tinh, vận tốc tối đa **60 km/h**, **Mức ưu tiên 2**. Kết nối Cẩm Lệ, Hòa Vang, Hòa Xuân, Thanh Khê.
- **03 Tuyến LRT Du Lịch:** LRT DL Tuyến phục vụ du lịch ven biển và dọc sông Hàn, vận tốc tối đa **40 km/h**, **Mức ưu tiên 3**. Kết nối Vịnh Đà Nẵng, Cầu Thuận Phước, Bãi biển Mỹ Khê và Phạm Văn Đồng.
- **34 Nhà ga kỹ thuật** và **hơn 50 cung ray liên kết** hai chiều phủ khắp địa bàn thành phố.

1.2. Giao diện người dùng tương tác hai bảng (Dual-Panel Interface)

Hệ thống loại bỏ hoàn toàn việc click chuột chọn ga mơ hồ trên bản đồ bằng việc cung cấp 2 bảng điều khiển chuyên biệt với kích thước khung mở rộng (bề rộng 460px, đạt 612px trên màn hình 2.5K) cùng phông chữ to rõ:

> **Bảng bên trái (Thiết lập & Lập lịch chuyến tàu):**
> - **Đề mục 1: Chọn loại tàu & Số hiệu:** 3 nút chọn loại tàu với 3 màu sắc chuẩn (MRT - Đỏ, LRT - Vàng cam, LRT DL - Xanh lá). Nhấp vào loại tàu sẽ mở hàng nút chọn số hiệu tương ứng (MRT-1..4, LRT-1..11, DL-1..4).
> - **Đề mục 2: Chọn ga đầu (Xuất phát):** Thẻ banner neon hiển thị ga xuất phát cùng danh sách cuộn 34 ga có tên đầy đủ, hỗ trợ con lăn chuột.
> - **Đề mục 3: Chọn ga cuối (Đích đến):** Thẻ banner neon hiển thị ga đích và tính toán lộ trình tối ưu ngay lập tức.
> - **Nút thao tác:** `[+ LƯU CHUYẾN TÀU NÀY (ENTER)]` đưa tàu vào hàng đợi; `[>> XUẤT PHÁT TẤT CẢ (SPACE)]` khởi hành đồng loạt theo thứ tự ưu tiên.

> **Bảng bên phải (Giám sát, Quản lý sự cố & Lịch sử cập bến):**
> - **Đề mục 1: Chọn ga bị tắc nghẽn / Sự cố:** Danh sách 34 ga kèm nút chuyển trạng thái `[BÌNH THƯỜNG]` và `[TẮC NGHẼN]`. Khi bị tắc nghẽn, ga trên bản đồ nhấp nháy đỏ kèm ký hiệu cảnh báo 'X', mạng lưới tự động cô lập điểm nghẽn.
> - **Đề mục 2: Danh sách tàu đã lưu:** Hiển thị chi tiết các chuyến tàu đã lưu trong hàng đợi, loại tàu, tuyến đường và nút xóa `[x]`.
> - **Đề mục 3: Bảng thông báo tàu nào đã đến nơi:** Nhật ký thời gian thực ghi nhận các đoàn tàu đã về đích an toàn kèm huy hiệu `[v]` và tên ga cập bến.

1.3. Hệ thống đường ray đôi 2 chiều (Double Track Bi-directional)

Mỗi liên kết giữa 2 nhà ga được mô hình hóa thành **2 đường ray song song độc lập** thay vì một đường nối đơn sơ:

- Với mỗi đoạn ray từ ga A sang ga B, vector pháp tuyến đơn vị sang bên phải chiều đi được tính toán: $\hat{n} = \left(-rac{dy}{L}, rac{dx}{L} ight)$.
- Làn ray $A ightarrow B$ được offset dạt sang phải một khoảng $w = 4.2 ext{ px}$. Ngược lại, làn ray $B ightarrow A$ dạt sang làn phải của hướng ngược lại. Hai làn chạy song song, cách nhau $pprox 8.4 ext{ px}$.
- Các thanh **tà-vẹt (sleepers)** nối vuông góc định kỳ mỗi $16 ext{ px}$ giữa 2 làn ray và **mũi tên chỉ hướng lưu thông** riêng biệt cho từng làn.
- Đoàn tàu khi chạy bám chính xác vào tâm làn ray bên phải, cho phép 2 tàu đi ngược chiều nhau trên cùng đoạn nối mà không bao giờ va chạm hay chạy lệch ra ngoài khoảng không.

1.4. Cơ chế chặn ray an toàn tự động (Mid-Journey & Departure Block Signaling)

Tuân thủ nguyên tắc an toàn đường sắt: **Không bao giờ cho phép 2 đoàn tàu cùng chạy trên một đoạn ray cùng chiều tại cùng một thời điểm**:

- **Khi khởi hành:** Nếu có 2 tàu cùng xuất phát từ một ga và đi cùng chặng ray đầu tiên, tàu có cấp ưu tiên cao hơn (MRT > LRT > LRT DL) sẽ xuất phát trước, tàu còn lại đỗ tại ga chờ.
- **Trong suốt hành trình (Mid-Journey):** Khi một tàu đến ga trung gian S_i và chuẩn bị tiến vào chặng ray $S_i ightarrow S_{i+1}$, nếu đoạn ray đang có tàu khác chạy, tàu đến sau **bắt buộc dừng đỗ tại ga S_i** với nhãn hổ phách `[CHO RAY: ID_Tàu_Trước]`.
- Khi tàu đi trước cập bến an toàn ga S_{i+1}, đoạn ray được giải phóng và tàu chờ tại S_i lập tức nhận tín hiệu xanh lăn bánh tiếp.

1.5. Cơ chế tự động tái định tuyến khi có sự cố (Dynamic Re-routing)

Khi người dùng kích hoạt `[TẮC NGHẼN]` cho một nhà ga:

- Toàn bộ tàu đang lưu thông có lộ trình qua ga đó sẽ tự động được hệ thống kích hoạt thuật toán `reRouteTrain()` để **tìm đường vòng thay thế** qua các nhánh ray liên thông khác.
- Nếu tìm được đường vòng: Tàu lập tức chuyển hướng sang nhánh mới mà không cần người dùng thao tác lại.
- Nếu mạng lưới bị cô lập hoàn toàn (không còn đường vòng tới đích): Tàu **bắt buộc dừng lại đỗ tại ga trước đó** với nhãn `[CHO GA NGHEN]`, **tuyệt đối không chạy vào ga đã bị tắc nghẽn**.

2. DANH SÁCH CÁC FILE, CƠ SỞ DỮ LIỆU & THUẬT TOÁN TƯƠNG ỨNG

Bảng dưới đây liệt kê chi tiết toàn bộ các tập tin mã nguồn trong dự án cùng cấu trúc dữ liệu và giải thuật tương ứng:

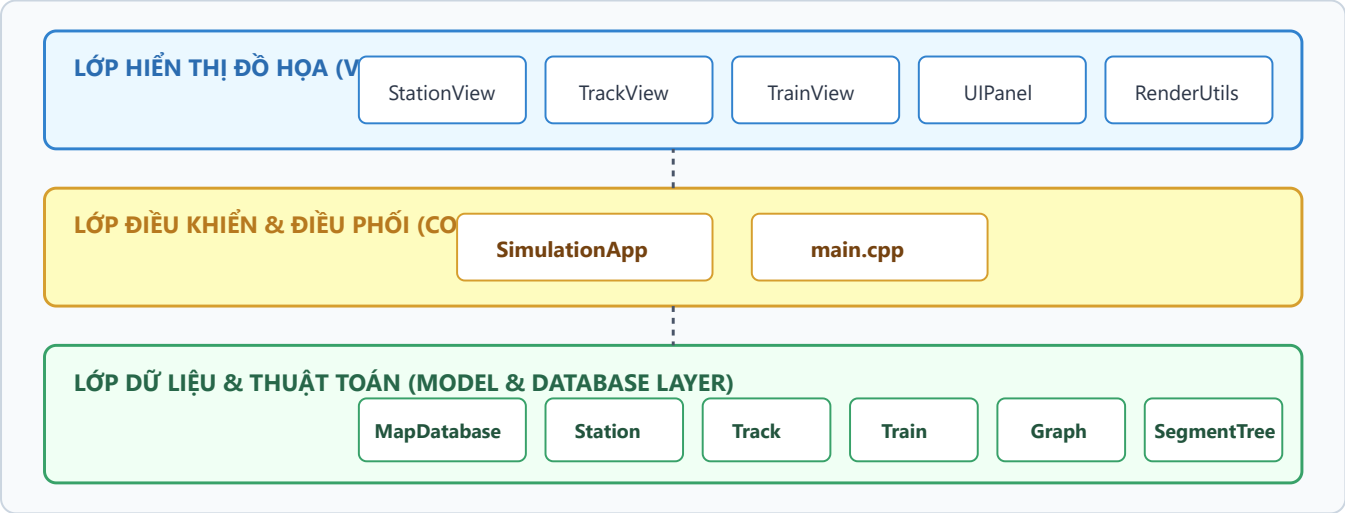
Tập tin	Tầng kiến trúc	Cấu trúc dữ liệu chính (CTDL)	Thuật toán cốt lõi & Cơ chế xử lý
`MapDatabase.h` `MapDatabase.cpp`	Database / Repository	• `struct StationData`: Tọa độ pixel (1038x1249), ID, Tên. • `struct TrackData`: srcId, destId, cự ly, tốc độ. • `std::vector<StationData>`, `std::vector<TrackData>`.	• **Cơ sở dữ liệu tĩnh chuẩn STL:** Nạp và quản lý 34 nhà ga quy hoạch và hơn 50 đoạn ray hai chiều TP. Đà Nẵng. • Độc lập với giao diện, phục vụ khởi tạo Graph và View.
`Station.h` `Station.cpp`	Model	• `string id`, `string name`. • `bool isBlocked` (Cờ báo tắc nghẽn/sự cố). • `std::vector<Track*> adjTracks` (**Danh sách kề - Adjacency List**).	• **Mô hình hóa đỉnh (Vertex):** Quản lý danh sách các cung ray xuất phát từ nhà ga với độ phức tạp truy xuất $O(1)$ amortized. • Cập nhật trạng thái cô lập khi xảy ra sự cố.
`Track.h` `Track.cpp`	Model	• `Station* source`, `Station* destination`. • `double distance` (km), `double speedLimit` (km/h). • `bool isOccupied` (Trạng thái chiếm dụng).	• **Mô hình hóa cung có hướng (Directed Edge):** Tính toán trọng số thời gian lưu thông cơ sở: $t = rac{ ext{distance}}{ ext{speedLimit}}$. • Thiết lập trạng thái chiếm dụng / đóng đường.
`Train.h` `Train.cpp`	Model	• `enum class TrainType` (MRT, LRT, LRT_TOURIST). • `int priority` (1, 2, 3). • `std::queue<Station*> route` (**Hàng đợi FIFO**). • `Station* destinationStation` (Ga đích cuối).	• **Mô hình hóa đoàn tàu & Lộ trình:** Vận hành hàng đợi FIFO quản lý các ga cần đi qua theo thứ tự. • Kiểm tra `containsStationInRoute()` phục vụ phát hiện ga sự cố để kích hoạt tái định tuyến.
`Graph.h` `Graph.cpp`	Model / Algorithm	• `std::unordered_map<string, Station*>`. • `std::vector<Track*>`. • `std::priority_queue` (**Min-Heap**) với custom comparator so sánh trọng số thời gian.	• **Thuật toán Dijkstra phân cấp ưu tiên (Priority Dijkstra):** Độ phức tạp $O((V + E) \log V)$. • **Priority Tuning:** Tàu MRT ưu tiên ray cao tốc ($ imes 0.85$), phạt ray du lịch ($ imes 2.5$); Tàu du lịch ưu tiên ray ngắm cảnh ($ imes 0.8$). • **Tự động tránh điểm nghẽn:** Bỏ qua toàn bộ ray có `isOccupied` hoặc `neighbor->isBlocked`.
`SegmentTree.h` `SegmentTree.cpp`	Model / Data Structure	• `std::vector<double> tree` (**Cây nhị phân dạng mảng kích thước $4N$**).	• **Cây phân đoạn (Segment Tree):** • Cập nhật trọng số phân đoạn trong $O(\log N)$. • Truy vấn tổng thời gian / cự ly

Tập tin	Tầng kiến trúc	Cấu trúc dữ liệu chính (CTDL)	Thuật toán cốt lõi & Cơ chế xử lý
		• Quản lý danh sách trọng số các phân đoạn ray.	trên đoạn $[L, R]$ bất kỳ trong $O(\log N)$ thay vì quét tuần tự $O(N)$.
`StationView.h` `StationView.cpp`	View	• `Vector2 pos`, `float radius`, `Color baseColor`. • `Station* model`. • Trạng thái `isHovered`, `isSelected`.	• **Trực quan hóa nhà ga:** Vẽ hình học tròn đa lớp, phân loại màu theo tốc độ cao nhất của tuyến kết nối. • **Hiệu ứng cảnh báo:** Nhấp nháy hào quang đỏ kèm ký hiệu 'X' cảnh báo khi ga bị sự cố. Phát hiện va chạm chuột `contains()`.
`TrackView.h` `TrackView.cpp`	View	• `StationView* srcView`, `StationView* destView`. • `Track* model`. • Vector 2D pháp tuyến đơn vị $\hat{n} = (-dy/L, dx/L)$.	• **Vẽ hệ thống đường ray đôi 2 chiều:** Dạt làn sang bên phải theo vector pháp tuyến với độ lệch $w = 4.2 ext{ px}$. • Vẽ hệ thống tà-vẹt (sleepers) vuông góc định kỳ mỗi $16 ext{ px}$ và mũi tên chỉ hướng lưu thông của riêng từng làn ray.
`TrainView.h` `TrainView.cpp`	View	• `Vector2 currentPos, targetPos`. • `std::vector<Vector2> trail` (**Circular Buffer vệt sáng**). • `Train* model`. • Trạng thái `isWaitingBlock`, `waitingOrder`.	• **Nội suy chuyển động mượt mà (Linear Interpolation):** Tàu di chuyển chuẩn tâm làn ray bên phải. • Vẽ thân tàu xoay theo góc chuyển động `atan2f(dy, dx)`, vệt sáng neon, nón đèn pha chiếu sáng. • **Offset xếp hàng chờ ray:** Lùi $18 ext{ px} imes ext{waitingOrder}$ tránh đè hình ảnh.
`UIPanel.h` `UIPanel.cpp`	View / UI	• Cấu trúc danh sách tĩnh cho các nút loại tàu, số hiệu. • Tọa độ hộp chữ nhật `Rectangle`, màu sắc Raylib. • Con trỏ tương tác `SimulationApp*`.	• **Dựng bảng điều khiển kính mờ hiện đại:** Bảng bên trái (Chọn tàu, ga đầu, ga cuối, nút Save, Run) và Bảng bên phải (Quản lý sự cố, danh sách đã lưu, nhật ký về đích). • Xử lý sự kiện nhấp chuột, cuộn chuột và hiệu ứng hover.
`RenderUtils.h` `RenderUtils.cpp`	View / UI Helper	• `Font gFont, gFontBold` (Segoe UI độ phân giải cao 64px). • Biến toàn cục `float gUiScale`.	• **Responsive UI Scaling:** Tự động tính tỷ lệ co giãn $gUiScale = rac{ ext{screenH}}{1080.0f}$ cho màn hình Full HD, 2.5K, 4K. • Tạo 10 tầng Mipmaps khử răng cưa trilinear cho phông chữ.

Tập tin	Tầng kiến trúc	Cấu trúc dữ liệu chính (CTDL)	Thuật toán cốt lõi & Cơ chế xử lý
`SimulationApp.h` `SimulationApp.cpp`	Controller / Engine	• `std::vector<QueuedDispatch> pendingFleet`. • `std::vector<ArrivedTrainInfo> arrivedFleet`. • `std::unordered_map<string, Vector2> stationPositions`.	• **Bộ điều phối trung tâm (Game Loop / Orchestrator):** Vận hành máy trạng thái, xử lý sự kiện bàn phím/chuột. • **Mid-journey Block Signaling:** Tự động dừng tàu khi chặng tiếp theo có tàu chạy, cấp phép giải phóng ray theo mức ưu tiên. • **Dynamic Re-routing:** Tái định tuyến thời gian thực khi có ga nghẽn.
`main.cpp`	Entry Point	• Đối tượng `SimulationApp app(1920, 1080)`.	• Điểm khởi chạy của chương trình, kích hoạt phương thức `app.run()` để bắt đầu vòng lặp mô phỏng.

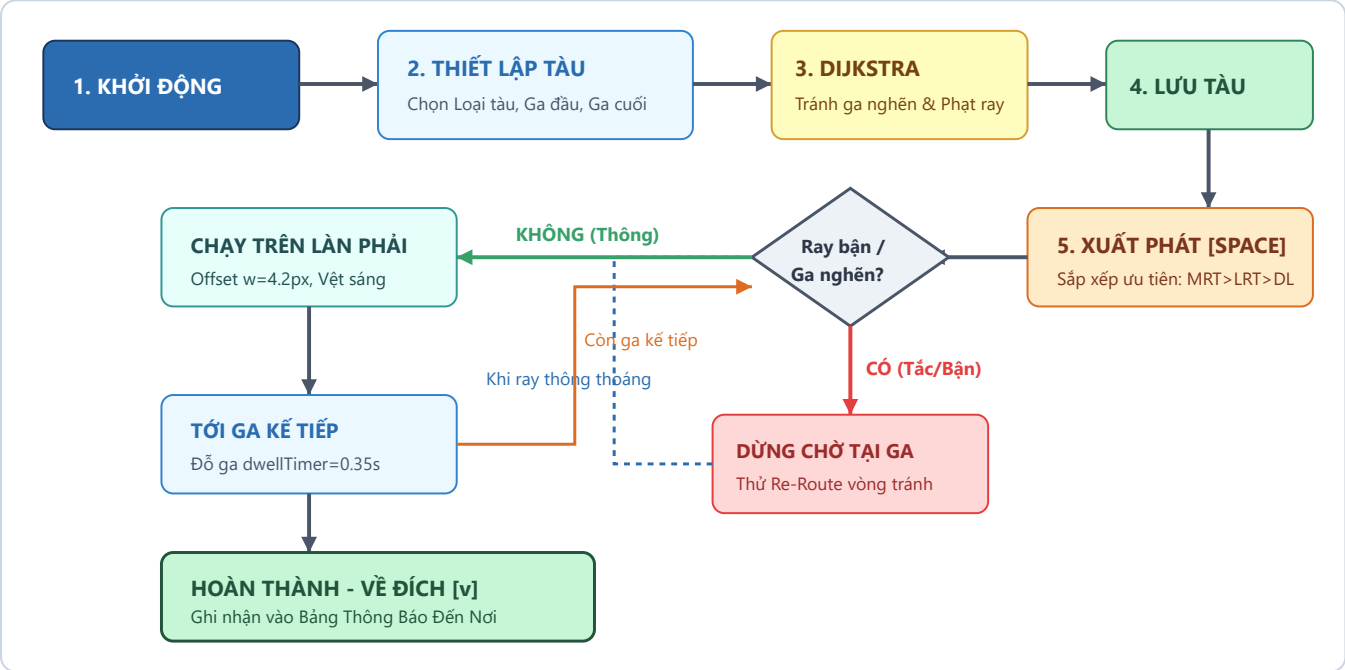
3. BIỂU ĐỒ TRỰC QUAN LUỒNG HOẠT ĐỘNG CỦA DỰ ÁN

3.1. Sơ đồ kiến trúc phân tầng hệ thống (Layered Architecture)



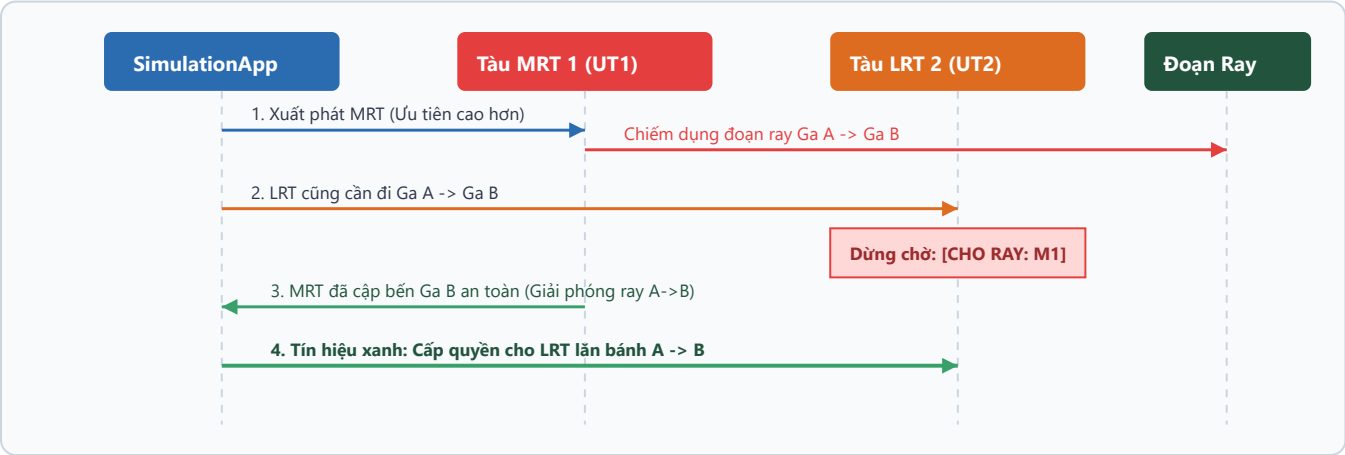
Hình 1: Mô hình phân tầng kiến trúc phần mềm Layered Architecture (MVC)

3.2. Sơ đồ luồng hoạt động chính của hệ thống điều phối (Operational Workflow)



Hình 2: Sơ đồ luồng trạng thái hoạt động và chu trình điều phối tàu khép kín

3.3. Sơ đồ tuần tự điều phối nhường ray giữa 2 tàu cùng tuyến (Block Signaling Sequence)



Hình 3: Sơ đồ trình tự điều phối chặn ray an toàn và nhường đường giữa hai đoàn tàu

4. HƯỚNG DẪN CÀI ĐẶT, BIÊN DỊCH & VẬN HÀNH

4.1. Yêu cầu môi trường & Thư viện phụ thuộc

- **Hệ điều hành:** Microsoft Windows 10 / 11 (64-bit).
- **Trình biên dịch:** MinGW UCRT64 / GCC hỗ trợ tiêu chuẩn **C++17** trở lên.
- **Công cụ dựng:** CMake 3.10+ và Make / Ninja.
- **Thư viện đồ họa:** Raylib 5.0 (liên kết cùng `opengl32`, `gdi32`, `winmm`).

4.2. Hướng dẫn biên dịch dự án từ mã nguồn

Thực hiện các lệnh sau trong terminal PowerShell tại thư mục gốc dự án:

```
# 1. Tạo thư mục build và cấu hình CMake
cmake -B build -G "MinGW Makefiles"

# 2. Biên dịch song song với MinGW
cmake --build build --parallel 4

# 3. Khởi chạy ứng dụng điều phối
./TrainDispatchSystem.exe
```

4.3. Bảng tra cứu phím tắt điều khiển nhanh

Phím tắt / Thao tác	Ý nghĩa & Chức năng điều phối tương ứng
`Chuột trái (Click)`	Chọn loại tàu, chọn số hiệu tàu, chọn ga đầu/ga cuối, bật/tắt sự cố ga, xóa tàu hoặc khởi hành hạm đội.
`Con lăn chuột (Wheel)`	Cuộn mượt mà danh sách ga đầu, danh sách ga cuối, danh sách ga sự cố, danh sách tàu đã lưu và lịch sử cập bến.
`[1]` / `[2]` / `[3]`	Chuyển đổi nhanh loại tàu: `[1]` MRT Cao Tốc, `[2]` LRT Đô Thị, `[3]` LRT Du Lịch.
`[ENTER]` / `[L]`	Lưu cấu hình đoàn tàu hiện tại vào danh sách hàng đợi (Saved Fleet) chuẩn bị xuất phát.
`[SPACE]` / `[X]`	**Xuất phát đồng loạt** toàn bộ các đoàn tàu trong danh sách chờ theo đúng thứ tự ưu tiên (MRT > LRT > DL).
`[C]`	Xóa sạch toàn bộ danh sách tàu đang xếp hàng chờ xuất phát.
`[R]`	**Reset toàn diện hệ thống:** Dọn dẹp toàn bộ tàu đang chạy, khôi phục trạng thái hoạt động bình thường cho mọi nhà ga.
`[M]`	Bật / Tắt lớp ảnh nền bản đồ vệ tinh TP. Đà Nẵng.
`[F11]` / `[F]`	Bật / Tắt chế độ hiển thị toàn màn hình (Fullscreen Responsive Viewport).